

WELL X Project Guide

1. What This Project Is

WELL X is a drilling-risk decision-support prototype for drilling engineers. It compares the current drilling state with historical incidents from nearby wells and produces explainable risk alerts.

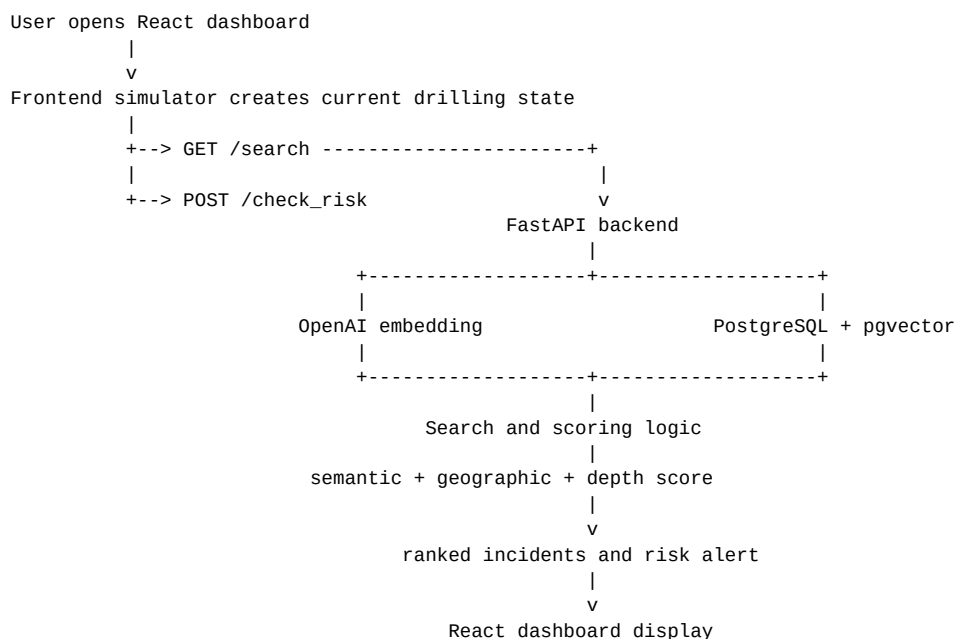
The project is a prototype, not an autonomous drilling-control system. It helps a human engineer make a better-informed decision.

The system combines four signals:

1. Semantic similarity: does the current situation mean something similar to an old incident-
2. Geographic proximity: is the historical well nearby-
3. Depth proximity: did the historical incident happen at a similar depth-
4. Explainability: which well, incident, depth, distance, and narrative caused the alert-

The current historical data is synthetic demo data generated by `data/generate_data.py`.

2. The Whole System in One Diagram



When PostgreSQL is unavailable, the backend reads `data/wells_data.json` directly and continues with geographic and depth ranking. That is the demo fallback.

3. Project Folders

```
WELL X/
|-- backend/
|   |-- .env                private local configuration
```

```

| |-- .env.example      safe configuration template
| |-- database.py       database connection
| |-- embeddings.py     OpenAI embedding creation
| |-- main.py          FastAPI routes
| |-- models.py         SQLAlchemy database model
| |-- requirements.txt  Python dependencies
| |-- search.py         hybrid search and scoring
| |-- seed_data.py     JSON-to-PostgreSQL importer
|
|-- data/
| |-- generate_data.py  deterministic synthetic-data generator
| |-- wells_data.json  generated historical well incidents
|
|-- frontend/
| |-- .env.example      frontend API URL template
| |-- index.html        browser HTML entry point
| |-- package.json      JavaScript dependencies and scripts
| |-- package-lock.json exact dependency versions
| |-- vite.config.js    Vite development configuration
| |-- src/
| | |-- App.jsx         main React dashboard
| | |-- main.jsx        React mounting entry point
|
|-- .gitignore          files that should not be committed
|-- README.md          short setup instructions
|-- docs/WELL_X_PROJECT_GUIDE.md this handbook

```

node_modules, dist, .venv, __pycache__, and generated cache files are environment/build output. They are not the core source code.

4. Backend File by File

4.1 `backend/database.py`

This is the database connection layer.

load_dotenv(...) loads variables from backend/.env.

DATABASE_URL is the PostgreSQL connection string. It tells SQLAlchemy which server, port, database, and credentials to use.

create_engine(...) creates the reusable SQLAlchemy database engine. pool_pre_ping=True checks that pooled connections are still alive before using them.

SessionLocal is a factory for database sessions. A session is used to read and write rows.

Base = declarative_base() is the parent used by the Well model.

The important resilience behavior is that a missing DATABASE_URL does not crash the entire import. Instead, engine and SessionLocal are left unavailable. The search layer can then use local JSON fallback data.

get_db() creates a session and closes it in a finally block. This prevents database connections from being left open.

Important term: ORM means Object-Relational Mapper. SQLAlchemy lets Python classes represent database tables instead of writing every SQL statement manually.

4.2 `backend/models.py`

This file defines the database table using SQLAlchemy.

The Well class represents one historical well incident. It maps to the PostgreSQL table named wells.

Important columns:

- id: unique database identifier.
- well_name: well name.
- field_name: field or basin name.
- latitude, longitude: geographic location.
- depth_start_m, depth_end_m: interval drilled by the well.
- incident_depth_m: depth where the incident occurred.
- formation_type: geological formation.
- incident_type: for example, Stuck Pipe or Gas Kick.
- severity: low, medium, or high.
- incident_date: date stored as a simple string for this prototype.
- narrative: full text description.
- embedding: 1,536-number vector used for semantic search.

Vector(1536) matches the output size of OpenAI's text-embedding-3-small model. PostgreSQL needs the pgvector extension to store and compare this column.

4.3 `backend/embeddings.py`

This file converts text into a numerical representation of meaning.

`build_embedding_text(well)` combines structured fields and the narrative:

```
Formation: Gas-Bearing Sand.  
Incident type: Stuck Pipe.  
Severity: medium.  
Narrative: ...
```

Including both metadata and narrative helps the embedding represent the complete incident context.

`get_embedding(text)` calls OpenAI's text-embedding-3-small model. The result is a vector of 1,536 floating-point values.

A vector is not a label. It is a numerical position in a high-dimensional meaning space. Texts with similar meaning tend to have vectors that are close together.

The function returns `None` when the API is not configured or fails. This creates degraded mode: the system can still rank records by location and depth.

4.4 `backend/seed\_data.py`

This is the data-loading script.

The process is:

1. Enable PostgreSQL's vector extension.

2. Create the wells table if necessary.
3. Check whether rows already exist.
4. Read data/wells_data.json.
5. Build embedding text for every record.
6. Request an embedding from OpenAI.
7. Insert metadata, narrative, and vector into PostgreSQL.
8. Commit the transaction.

The script skips seeding when the table already contains records. This prevents accidental duplicates.

Run from the project root:

```
.\.venv\Scripts\python.exe backend\seed_data.py
```

A real database and valid connection string are required for this script. The API fallback does not replace database seeding for the full PostgreSQL path.

4.5 `backend/search.py`

This is the main ranking engine.

The configured weights are:

```
semantic similarity: 50 percent
geographic proximity: 30 percent
depth proximity:    20 percent
```

The final score is:

```
combined = 0.5 * semantic + 0.3 * geographic + 0.2 * depth
```

Haversine distance

`haversine_km(...)` calculates the distance between two latitude/longitude points while accounting for the Earth's curvature.

Geographic score

The maximum useful radius is 100 km:

```
geo_score = max(0, 1 - distance_km / 100)
```

A well at 0 km has a score of 1.0. A well at 50 km has a score of about 0.5. A well at 100 km or farther has a score of 0.

Depth score

The tolerance is 500 m:

```
depth_score = max(0, 1 - depth_difference / 500)
```

An incident at the same depth scores 1.0. An incident 250 m away scores 0.5. An incident 500 m or farther away scores 0.

Search process

search_similar_incidents(...) does the following:

1. Create an embedding for the query.
2. If PostgreSQL and the embedding are available, retrieve the most semantically similar records with pgvector.
3. If PostgreSQL is unavailable, read the generated JSON file.
4. Calculate semantic, geographic, and depth scores.
5. Sort results from highest combined score to lowest.
6. Return only the requested top_k records.

The local fallback uses semantic score 0 and still uses geographic and depth scores. That makes the demo usable without a database, but semantic ranking is stronger when PostgreSQL and OpenAI are available.

4.6 `backend/main.py`

This is the FastAPI application layer.

FastAPI(...) creates the web API.

CORS means Cross-Origin Resource Sharing. It allows the frontend, which runs on port 5173, to call the backend, which runs on port 8000. The allowed origins are configurable with CORS_ORIGINS.

GET /health

Returns service status, database status, and embedding configuration status.

Example:

```
{
  "status": "degraded",
  "service": "well-x-api",
  "database": "unavailable",
  "embeddings": "configured"
}
```

degraded is truthful: the API process is alive, but a dependency is unavailable.

GET /search

Query parameters:

- lat: current latitude.
- lon: current longitude.
- depth_m: current drilling depth.
- query: free-text question.
- top_k: number of results.

It returns the query and ranked historical incidents.

POST /check_risk

This is the live-feed endpoint. The request body contains latitude, longitude, current depth, planned formation, and top_k.

Because a live feed may not contain free text, the backend constructs a semantic query such as:

```
Drilling incident such as stuck pipe, lost circulation,  
gas kick or wellbore instability in Gas-Bearing Sand  
formation at around 2500 metres depth
```

The endpoint finds similar incidents, reads the highest combined score, and converts it into an alert level:

- HIGH: score at least 0.65
- MEDIUM: score at least 0.50
- LOW: score at least 0.35
- NONE: below the low threshold

It also returns only results above the low threshold as active alerts.

4.7 `backend/requirements.txt`

These dependencies provide the backend technology:

- fastapi: web API framework.
- uvicorn: development ASGI server.
- sqlalchemy: database ORM.
- psycopg2-binary: PostgreSQL driver.
- pgvector: vector support for SQLAlchemy/PostgreSQL.
- openai: OpenAI API client.
- python-dotenv: .env configuration loading.

5. Data Files

5.1 `data/generate\_data.py`

This is a deterministic synthetic-data generator.

It defines geographic clusters such as North Bank Field, Central Assam Field, Western Desert Field, and Offshore Bay Block.

It defines incident archetypes such as:

- Stuck Pipe
- Lost Circulation
- Gas Kick
- Wellbore Instability

- H2S Gas Show
- Drill String Washout
- Shallow Water Flow

Each archetype contains:

- Possible formations.
- A depth range.
- Severity choices.
- Narrative sentence templates.
- Random operational values.
- Recommendations.

`random.Random(42)` makes the output deterministic. Running the generator again with the same count creates repeatable demo data.

Run:

```
.\.venv\Scripts\python.exe data\generate_data.py
```

The default output is 60 records.

5.2 `data/wells\_data.json`

This is the generated historical dataset. Each record contains 12 fields required by the backend model.

The current dataset has:

- 60 records.
- 7 incident types.
- 14 low-severity records.
- 31 medium-severity records.
- 15 high-severity records.
- Incident depths from 285 m to 3341 m.
- Dates from 2017 to 2024.

It is synthetic data, so it demonstrates the architecture but is not evidence about real Oil India incidents.

6. Frontend File by File

6.1 `frontend/src/main.jsx`

This is the React entry point. It finds the HTML element with id root and renders the App component inside React Strict Mode.

6.2 `frontend/src/app.jsx`

This is the main dashboard file.

It contains the UI components, mock simulator, state, backend calls, and interaction logic.

Design tokens

The C object stores colors used across the dashboard. This keeps the visual theme consistent.

API configuration

```
const API_BASE = import.meta.env.VITE_API_BASE || "http://127.0.0.1:8000";
```

The frontend can use a deployment URL through VITE_API_BASE. It defaults to the local backend for development.

Live monitor

The live monitor displays depth, ROP, torque, and mud flow. setInterval advances the simulated depth every 1.5 seconds. genPoint adds controlled random variation.

Recharts renders the ROP and torque charts.

Offset map

The map is a custom SVG rather than a tile service. It shows the current well and nearby offset wells. The 2 km, 5 km, and 10 km controls filter visible wells.

Clicking or hovering a pin shows the formation, hazard, severity, distance, and detail.

Alerts panel

The alerts panel shows:

- Hazard title.
- Source well.
- Confidence percentage.
- Recommended action.
- Explainability text.
- Comparison chart.
- Feedback buttons.
- Dismiss button.

The frontend can display locally simulated alerts and backend alerts. Backend alerts are marked internally with api: true and replace previous backend alerts on the next risk check.

Backend search integration

When the search query contains at least two characters, the frontend calls /search after a short debounce. It maps backend results into the table format used by the Report Search component.

If the backend is unavailable, the screen falls back to the local REPORT_DB mock records.

Backend risk integration

Every few seconds during the simulated feed, the frontend calls `/check_risk` with the current location and depth. Successful results become alerts. A failed request changes the connection display to degraded and leaves the local simulator available.

Report search

The report search table filters records by well, field, value, and depth. It supports sorting and human correction. Corrections are kept in React state and written to the audit log.

This is a human-in-the-loop design: the AI-style extraction is not treated as unquestionable.

Risk timeline

The timeline divides the depth range into bands and colors bands that overlap known danger zones. Hover text explains the flagged incident.

Audit trail

The audit trail records alerts shown, alerts dismissed, corrections, and feedback actions. The current implementation keeps this in memory for the browser session.

Connection indicator

The indicator displays Good, Degraded, or Offline. In degraded or offline mode, the UI communicates that information may be last-known or fallback data.

6.3 `frontend/package.json`

Defines the frontend project and commands:

```
npm run dev
npm run build
npm run preview
```

Main libraries:

- React and React DOM.
- Recharts for charts.
- Lucide React for icons.
- Vite and the React plugin for development/building.

6.4 `frontend/vite.config.js`

Configures Vite, enables the React plugin, uses port 5173, and requires that port to be free with `strictPort`.

6.5 `frontend/index.html`

The browser shell. It contains the root element where React mounts the application.

7. End-to-End Example

Suppose the frontend sends:

```
{
  "lat": 27.35,
  "lon": 95.15,
  "current_depth_m": 2500,
  "planned_formation": "Gas-Bearing Sand",
  "top_k": 5
}
```

The backend:

1. Builds a risk query.
2. Requests an embedding if OpenAI is configured.
3. Queries pgvector if PostgreSQL is available.
4. Otherwise loads the local JSON records.
5. Calculates distance from the current well to each historical well.
6. Calculates distance between current depth and incident depth.
7. Combines the scores.
8. Filters results above the LOW threshold.
9. Returns an alert level and explainable results.

The frontend then:

1. Converts each result into an alert card.
2. Shows the source well and confidence.
3. Shows the narrative explanation.
4. Shows the recommended action.
5. Allows feedback or dismissal.
6. Records the engineer action in the audit trail.

8. Important Terms

- API: a contract that lets software communicate.
- FastAPI: the Python framework serving the endpoints.
- React: the JavaScript library used for the dashboard.
- Vite: the frontend development and build tool.
- SQLAlchemy: Python ORM for database access.
- PostgreSQL: relational database used for historical incidents.
- pgvector: PostgreSQL extension for vector storage and similarity search.
- Embedding: a numerical representation of text meaning.
- Semantic similarity: similarity of meaning rather than exact words.
- Haversine formula: geographic distance calculation using coordinates.

- CORS: browser security configuration allowing approved frontend origins.
- ORM: mapping database tables to Python objects.
- JSON: structured text format used for generated data and API messages.
- Pydantic model: request validation model used by FastAPI.
- Fallback: alternate behavior used when a preferred service is unavailable.
- Degraded mode: the system remains usable with fewer capabilities.
- Top-k: the number of highest-ranked results returned.
- Human-in-the-loop: a human can review, correct, accept, or reject system output.
- Confidence: an estimate shown to the user, not a guarantee of truth.

9. What Is Complete

- Synthetic data generation.
- JSON validation.
- PostgreSQL model definition.
- Optional OpenAI embeddings.
- Vector search path.
- Geographic and depth reranking.
- Local JSON fallback.
- FastAPI API endpoints.
- React live monitor.
- Offset map.
- Explainable alerts.
- Report search and correction flow.
- Risk timeline.
- Audit trail.
- Feedback controls.
- Connection status.
- Configurable environment settings.
- Project README and ignore rules.

10. What Still Requires Real Infrastructure

The code can run in degraded mode, but the full production-like path requires:

1. A real PostgreSQL or Supabase DATABASE_URL.
2. The pgvector extension enabled.
3. The generated records seeded into PostgreSQL.
4. A valid OpenAI API key for semantic embeddings.
5. Real incident data and calibrated thresholds before production use.

6. Authentication and authorization before exposing the API publicly.
7. Persistent audit and feedback storage instead of React memory.
8. Real telemetry integration instead of the simulated drilling feed.

Never present synthetic records as real field history. Explain that the dataset demonstrates the workflow and that production data would be imported using the same schema.

11. Commands Cheat Sheet

From the project root:

```
# Validate generated Python source
.\.venv\Scripts\python.exe -m py_compile data\generate_data.py

# Regenerate synthetic data
.\.venv\Scripts\python.exe data\generate_data.py

# Install backend dependencies
.\.venv\Scripts\python.exe -m pip install -r backend\requirements.txt

# Seed the database
.\.venv\Scripts\python.exe backend\seed_data.py
```

Backend terminal:

```
cd backend
..\venv\Scripts\python.exe -m uvicorn main:app --reload
```

Frontend terminal:

```
cd frontend
npm install
npm run dev
```

Validation:

```
npm run build
```

URLs:

- Frontend: <http://127.0.0.1:5173>
- API docs: <http://127.0.0.1:8000/docs>
- Health: <http://127.0.0.1:8000/health>

12. Short Explanation for Judges

WELL X is an explainable drilling-risk decision-support prototype. It compares live drilling conditions with historical incidents from nearby wells. The system uses embeddings to compare meaning, geographic distance to compare nearby wells, and depth distance to compare similar drilling intervals. These signals are combined into a transparent score. The dashboard shows the source incident, distance, depth, confidence, explanation, recommended action, and feedback controls. The prototype uses synthetic data, but the architecture is ready for real historical reports and live drilling feeds.